

Kubernetes Event Triage: SRE / DevOps Challenge

Problem Statement

A modern observability stack ingests Kubernetes events as OpenTelemetry logs via the OTel Collector's `k8seventsreceiver`. In this dataset, each individual event occurrence is its own log record (`k8s.event.count: 1`). A busy cluster produces tens of thousands of these records per hour, the vast majority of them normal lifecycle noise (`Pulled`, `Created`, `Started`). Your tool must coalesce them itself. Real pipelines emit either one log per occurrence (watch-based) or repeated logs as the API server's `count` increments, and the tool has to group in either case.

When something breaks, the on-call engineer needs the actual failure surfaced from 20,000 log lines.

This challenge has two parts:

1. **Build a triage tool** that ingests OTel K8s event logs, diagnoses the failure pattern, and produces a prioritized human-readable summary.
2. **Use your tool to diagnose three unknown failure scenarios** and write up your findings.

We grade your code and your on-call reasoning.

Part 1: Build the Tool

A CLI that:

1. Reads a JSONL file of OTel log records (~20,000 records, one per line).
 2. Filters out informational noise.
 3. **Groups related records itself.** Events are independent in OTel form, so your tool must coalesce repeated occurrences (same object, same reason) into a single finding with count and time range.
 4. **Diagnoses each finding.** Classify severity, identify the likely cause, and surface the pattern (sustained crash-loop, transient blip, deploy-correlated failure, capacity issue, node issue, etc.). A summary that lists counts without an interpretation is half the work.
 5. Outputs a prioritized human-readable summary.
-

Input Format

JSON Lines. Each line is one OTel log record. The schema follows the OTel Collector's `k8seventsreceiver` output (plus the usual downstream `resourcedetection` / `k8sattributes` enrichment for cluster name). Each occurrence in the cluster is its own line with its own millisecond-precision timestamp. If the kubelet emits 47 `Unhealthy` events for one pod, that's 47 lines.

Sample record (one line, formatted for readability here):

```

{
  "timestamp": "2024-06-01T10:15:23.456Z",
  "observed_timestamp": "2024-06-01T10:15:23.502Z",
  "severity_text": "Warning",
  "severity_number": 13,
  "body": "Back-off restarting failed container checkout in pod checkout-service-7d4f8b9c5-abc12_production(8c5a8e6f-1234-5678-9abc-def012345678)",
  "attributes": {
    "k8s.event.reason": "BackOff",
    "k8s.event.action": "BackOff",
    "k8s.event.start_time": "2024-06-01T10:14:11.000Z",
    "k8s.event.name": "checkout-service-7d4f8b9c5-abc12.18a673b1629e0e76",
    "k8s.event.uid": "416d4495-3040-403a-b212-4702f4989d65",
    "k8s.event.count": 1,
    "k8s.namespace.name": "production"
  },
  "resource": {
    "k8s.cluster.name": "prod-us-east-1",
    "k8s.node.name": "node-3",
    "k8s.object.kind": "Pod",
    "k8s.object.name": "checkout-service-7d4f8b9c5-abc12",
    "k8s.object.uid": "8c5a8e6f-1234-5678-9abc-def012345678",
    "k8s.object.fieldpath": "spec.containers(checkout)",
    "k8s.object.api_version": "v1",
    "k8s.object.resource_version": "154269805",
    "service.name": "k8s-events"
  }
}

```

Object identity (`k8s.object.*` , `k8s.node.name`) lives under **resource** . Event metadata (`k8s.event.*` , `k8s.namespace.name`) lives under **attributes** . This matches the real receiver.

The fields your tool will care about most:

- `timestamp` : when the event happened in the cluster.
- `severity_text` : Normal for K8s Normal events, Warning for K8s Warning events.
- `attributes["k8s.event.reason"]` : the event reason (the most important field).
- `resource["k8s.object.name"]` , `resource["k8s.object.kind"]` , `attributes["k8s.namespace.name"]` : what the event is about.
- `resource["k8s.node.name"]` : the node, when the event was emitted by a kubelet (absent for scheduler/controller events).
- `body` : the human-readable message. For Failed and BackOff , the body tells you whether the failure is image-pull or container-restart.

There is no `k8s.event.source.component` attribute. Attribute events to controllers by `reason` and `k8s.object.kind` :

- Pulled / Created / Started / Killing / BackOff / Unhealthy / OOMKilling / Evicted / Failed on a Pod : kubelet.
- Scheduled / FailedScheduling on a Pod : default-scheduler .
- SuccessfulCreate / SuccessfulDelete on a ReplicaSet : replicaset-controller .
- ScalingReplicaSet on a Deployment : deployment-controller .
- NodeNotReady / NodeHasDiskPressure on a Node : node-controller .

Event reasons your tool should handle

Reason	severity_text	Body looks like	What it means	Severity
BackOff	Warning	Back-off restarting failed container ...	Container is crash-looping	Critical
BackOff	Normal	Back-off pulling image "..."	Repeated image-pull retry	Critical (in image-pull context)
Failed	Warning	Failed to pull image ... , Error: ErrImagePull , Error: ImagePullBackOff	Image pull failed	Critical
Failed / FailedCreatePodSandBox	Warning	other	Container or pod sandbox creation failed	Critical
OOMKilling	Warning	Memory cgroup out of memory ...	Out-of-memory kill	Critical
FailedScheduling	Warning	0/N nodes are available: ...	Pod can't be placed (resources/taints)	Warning
Unhealthy	Warning	Liveness probe failed: ... or Readiness probe failed: ...	Probe failing; body says which	Warning
FailedMount	Warning	MountVolume.SetUp failed for ...	Volume mount failure	Warning

Reason	severity_text	Body looks like	What it means	Severity
Evicted	Warning	The node had condition: [DiskPressure].	Pod evicted (node pressure)	Warning
NodeNotReady / NodeHasDiskPressure	Warning	node condition message	Node went unhealthy	Critical
Pulled , Created , Started , Scheduled , Killing , SuccessfulCreate , SuccessfulDelete	Normal	n/a	Normal lifecycle	Ignore

Note: ImagePullBackOff and ErrImagePull are not Kubernetes event *reasons*. They appear in the **body** of Failed events. Your tool should distinguish image-pull failures from other Failed events by reading the body.

You may extend this list. We care that you classify sensibly, not that you cover every reason.

Expected Output

Human-readable summary on stdout. Occurrence counts and time ranges are computed by your tool from the independent log records. The example below illustrates the *format and level of detail* expected. The actual reasons, services, and counts you report will depend on the file:

```
=====
KUBERNETES CLUSTER EVENT TRIAGE
Records ingested: 20,000 | Filtered as noise: 19,247 | Issues: 2
=====

[CRITICAL] <Reason> on <service-name> (<namespace>)
<one-line scope summary: how many pods, over what time range>
<evidence: event counts, signatures, patterns observed>
Likely cause: <short rationale grounded in the evidence>

[WARNING] <Reason> on <other-service> (<namespace>)
<scope summary>
Likely cause: <short rationale>
```

A `--json` flag for structured output is optional but recommended. You'll find it useful for your own analysis in Part 2.

Part 2: Diagnose the Test Scenarios

You will receive **six JSONL files**. Three have known answers (described below) for you to develop and validate your tool against. The other three are **opaque test scenarios**. Your job is to run your tool on them, read the output, and write a short diagnosis for each.

This is the on-call test. We want to see how you reason from your tool's output to a conclusion.

A single root cause can spread across multiple services, namespaces, or nodes. Seemingly unconnected findings are sometimes side effects of the same underlying issue. Look for shared dimensions across findings (the same node, the same time window, the same upstream dependency) before treating any pair of issues as independent.

Provided scenarios (known answers, use for development)

File	Scenario
01-healthy.jsonl	Healthy cluster baseline. No real failure (though a few unrelated transient warnings exist). Your tool should output "no issues detected" and exit 0.
02-memory-leak.jsonl	One service is repeatedly OOMKilling due to a memory leak. Your tool should identify the service, the OOM signature, and the sustained crash-loop pattern.
03-image-pull-failure.jsonl	A deployment references a missing image tag, and new pods cannot pull. Your tool should identify the affected service and the ImagePullBackOff pattern.

Test scenarios (you diagnose)

File	Scenario
04-test-a.jsonl	Run your tool. Diagnose.
05-test-b.jsonl	Run your tool. Diagnose.
06-test-c.jsonl	Run your tool. Diagnose.

For each test scenario, your **Analysis Report** must include:

- **Diagnosis:** what is broken? Be specific (service, resource, failure mode, scope).
- **Evidence:** which events from your tool's output led you there? Quote specific findings.
- **Suggested remediation:** what would you do in the next 5 minutes if this paged you?
- **Confidence:** how sure are you (high / medium / low), and what would raise it?

~150 words per scenario is plenty. We grade primarily on reasoning quality. Ground each claim in specific events from your tool's output, and state what would raise your confidence.

Acceptance Criteria

- ☐ Parses OTel-format JSONL records correctly at 20K scale.
- ☐ Filters out normal lifecycle events.
- ☐ **Groups independent records** by `resource["k8s.object.*"]`, `attributes["k8s.namespace.name"]`, and `attributes["k8s.event.reason"]`, and computes occurrence count + time range from the records themselves.
- ☐ Outputs reason, affected objects, occurrence count, time range, severity, likely cause.
- ☐ Severity classification matches the table above (or is defensibly different).
- ☐ No false positives on `01-healthy.jsonl`.
- ☐ Correctly identifies the primary issue in `02-memory-leak.jsonl` and `03-image-pull-failure.jsonl`.
- ☐ Distinguishes transient one-off warnings from sustained problems.
- ☐ Processes a 20K-record file in under 5 seconds.
- ☐ Analysis Report covers all three test scenarios (04, 05, 06).

Evaluation

Criterion	Weight
Analysis Report (SRE judgment on the three test scenarios)	35%
Tool correctness on the three provided scenarios	25%
Event interpretation and coalescing logic (right grouping keys, sensible time windows)	15%
Output clarity and noise discrimination	10%
Code quality and tests	10%
Documentation	5%

The weighting reflects our priorities. We grade primarily on the quality of your reasoning and the readability of your tool's output. Code quality and tests are secondary criteria.

Deliverables

1. **Source code** with build/run instructions.
2. **README** with usage and example output for one provided scenario.
3. **Tests** covering the three provided scenarios.
4. **Analysis Report** (`ANALYSIS.md`): your diagnosis for scenarios 04, 05, and 06.
5. **Raw tool output** for each of the three test scenarios. This is required, not optional. Capture stdout (and `--json` output if you implemented it) to files and include them in the submission.

Time Expectation

4 days to submission. Manage your own time. We don't track hours. We grade the tool you deliver and the quality of your analysis.

Focus on a tight, well-reasoned tool and a substantial analysis report.

AI assistants and libraries are encouraged. We grade on your understanding of Kubernetes event semantics in an OTel-shaped world, your ability to turn a wall of independent log records into something useful, and how well you read your own output to do real on-call work.